

```

def import_excel(self):
    file_path = filedialog.askopenfilename(filetypes=[("Excel files",
    "*.xlsx *.xls")])
    if file_path:
        try:
            df = pd.read_excel(file_path)
            for index, row in df.iterrows():
                student =
db_session.query(Student).filter_by(student_id=row['Cédula']).first()
                if student:
                    student.full_name = row['Nombre Completo']
                    student.email = row['Correo Electrónico']
                    student.program = row['Programa Académico']
                    student.campus = row['Sede']
                else:
                    new_student = Student(
                        full_name=row['Nombre Completo'],
                        student_id=row['Cédula'],
                        email=row['Correo Electrónico'],
                        program=row['Programa Académico'],
                        campus=row['Sede']
                    )
                    db_session.add(new_student)
                db_session.commit()
                messagebox.showinfo("Éxito", "Datos importados correctamente
desde Excel")
            except IntegrityError:
                db_session.rollback()
                messagebox.showerror("Error", "Error de integridad en la
importación")

```

1. Función import_excel(self)

def import_excel(self):

- Declara un método en una clase. Este método permite importar datos desde un archivo de Excel y guardarlos en una base de datos.

2. Abrir un cuadro de diálogo para seleccionar un archivo Excel

```
file_path = filedialog.askopenfilename(filetypes=[("Excel files", "*.xlsx *.xls")])
```

- Usa `filedialog.askopenfilename()` para abrir una ventana emergente que permita al usuario seleccionar un archivo.
 - Solo se pueden seleccionar archivos con extensión `.xlsx` y `.xls` (formatos de Excel).
-

3. Verificar si se seleccionó un archivo

if `file_path`:

- Si el usuario seleccionó un archivo (es decir, `file_path` no está vacío), entonces el código continúa ejecutándose.
-

4. Intentar leer el archivo Excel

try:

```
df = pd.read_excel(file_path)
```

- Usa `pandas (pd.read_excel(file_path))` para leer el contenido del archivo Excel en un `DataFrame (df)`.
-

5. Iterar sobre las filas del DataFrame

for `index, row` in `df.iterrows()`:

- Usa `iterrows()` para recorrer cada fila del `DataFrame` y extraer los datos de cada estudiante.
-

6. Buscar si el estudiante ya existe en la base de datos

```
student = db_session.query(Student).filter_by(student_id=row['Cédula']).first()
```

- Busca en la base de datos si ya existe un estudiante con la **Cédula** de la fila actual del Excel.
- `db_session.query(Student).filter_by(student_id=row['Cédula']).first()` devuelve el primer estudiante encontrado o `None` si no existe.

7. Actualizar datos si el estudiante ya existe

if student:

```
student.full_name = row['Nombre Completo']
```

```
student.email = row['Correo Electrónico']
```

```
student.program = row['Programa Académico']
```

```
student.campus = row['Sede']
```

- Si el estudiante **ya existe**, actualiza sus datos con los valores de la fila de Excel.

8. Crear un nuevo estudiante si no existe en la base de datos

else:

```
new_student = Student(
```

```
    full_name=row['Nombre Completo'],
```

```
    student_id=row['Cédula'],
```

```
    email=row['Correo Electrónico'],
```

```
    program=row['Programa Académico'],
```

```
    campus=row['Sede']
```

```
)
```

```
db_session.add(new_student)
```

- Si el estudiante **no existe**, crea una nueva instancia de la clase Student con los datos de la fila de Excel.
- Luego, agrega el nuevo estudiante a la sesión de base de datos (db_session.add(new_student)), pero **aún no se guarda en la base de datos**.

9. Confirmar cambios en la base de datos

```
db_session.commit()
```

- Guarda todos los cambios en la base de datos.
-

10. Mostrar mensaje de éxito

`messagebox.showinfo("Éxito", "Datos importados correctamente desde Excel")`

- Muestra un mensaje de confirmación indicando que la importación fue exitosa.
-

11. Manejar errores de integridad en la base de datos

`except IntegrityError:`

`db_session.rollback()`

`messagebox.showerror("Error", "Error de integridad en la importación")`

- Si ocurre un error de integridad (por ejemplo, intentar insertar un estudiante con una Cédula duplicada que tiene una restricción UNIQUE en la base de datos), se revierte la transacción con `db_session.rollback()`.
 - Luego, muestra un mensaje de error al usuario.
-

◆ Resumen del funcionamiento

1. Abre un cuadro de diálogo para seleccionar un archivo Excel.
2. Lee el archivo usando pandas.
3. Itera sobre las filas del archivo:
 - Si el estudiante ya existe en la base de datos, actualiza sus datos.
 - Si no existe, lo agrega como un nuevo registro.
4. Guarda los cambios en la base de datos.
5. Muestra un mensaje de éxito o de error si hay problemas de integridad.

Si necesitas mejorar este código o agregar más validaciones, dime y lo ajustamos. 🚀

Claro, te explico línea por línea lo que hace este código en Python:

1. Función `import_excel(self)`

```
def import_excel(self):
```

- Declara un método en una clase. Este método permite importar datos desde un archivo de Excel y guardarlos en una base de datos.
-

2. Abrir un cuadro de diálogo para seleccionar un archivo Excel

```
file_path = filedialog.askopenfilename(filetypes=[("Excel files", "*.xlsx *.xls")])
```

- Usa `filedialog.askopenfilename()` para abrir una ventana emergente que permita al usuario seleccionar un archivo.
 - Solo se pueden seleccionar archivos con extensión `.xlsx` y `.xls` (formatos de Excel).
-

3. Verificar si se seleccionó un archivo

```
if file_path:
```

- Si el usuario seleccionó un archivo (es decir, `file_path` no está vacío), entonces el código continúa ejecutándose.
-

4. Intentar leer el archivo Excel

```
try:
```

```
df = pd.read_excel(file_path)
```

- Usa `pandas (pd.read_excel(file_path))` para leer el contenido del archivo Excel en un `DataFrame (df)`.
-

5. Iterar sobre las filas del DataFrame

```
for index, row in df.iterrows():
```

- Usa `iterrows()` para recorrer cada fila del `DataFrame` y extraer los datos de cada estudiante.
-

6. Buscar si el estudiante ya existe en la base de datos

```
student = db_session.query(Student).filter_by(student_id=row['Cédula']).first()
```

- Busca en la base de datos si ya existe un estudiante con la **Cédula** de la fila actual del Excel.
 - `db_session.query(Student).filter_by(student_id=row['Cédula']).first()` devuelve el primer estudiante encontrado o `None` si no existe.
-

7. Actualizar datos si el estudiante ya existe

if student:

```
student.full_name = row['Nombre Completo']
```

```
student.email = row['Correo Electrónico']
```

```
student.program = row['Programa Académico']
```

```
student.campus = row['Sede']
```

- Si el estudiante **ya existe**, actualiza sus datos con los valores de la fila de Excel.
-

8. Crear un nuevo estudiante si no existe en la base de datos

else:

```
new_student = Student(  
    full_name=row['Nombre Completo'],  
    student_id=row['Cédula'],  
    email=row['Correo Electrónico'],  
    program=row['Programa Académico'],  
    campus=row['Sede']  
)
```

```
db_session.add(new_student)
```

- Si el estudiante **no existe**, crea una nueva instancia de la clase `Student` con los datos de la fila de Excel.

- Luego, agrega el nuevo estudiante a la sesión de base de datos (`db_session.add(new_student)`), pero **aún no se guarda en la base de datos**.
-

9. Confirmar cambios en la base de datos

`db_session.commit()`

- Guarda todos los cambios en la base de datos.
-

10. Mostrar mensaje de éxito

`messagebox.showinfo("Éxito", "Datos importados correctamente desde Excel")`

- Muestra un mensaje de confirmación indicando que la importación fue exitosa.
-

11. Manejar errores de integridad en la base de datos

`except IntegrityError:`

`db_session.rollback()`

`messagebox.showerror("Error", "Error de integridad en la importación")`

- Si ocurre un error de integridad (por ejemplo, intentar insertar un estudiante con una Cédula duplicada que tiene una restricción `UNIQUE` en la base de datos), se revierte la transacción con `db_session.rollback()`.
 - Luego, muestra un mensaje de error al usuario.
-

◆ Resumen del funcionamiento

1. Abre un cuadro de diálogo para seleccionar un archivo Excel.
2. Lee el archivo usando `pandas`.
3. Itera sobre las filas del archivo:
 - Si el estudiante ya existe en la base de datos, actualiza sus datos.
 - Si no existe, lo agrega como un nuevo registro.

4. Guarda los cambios en la base de datos.
5. Muestra un mensaje de éxito o de error si hay problemas de integridad.